

Data Science Programmmentwurf

- Kurs: TINF22B
- Matrikelnummern: 6732014, 7191637

1. Business Understanding

1.1 Kontext

Sie sind Remote Worker in der Kreativbranche und möchten für 6 Monate mit einem Freund / einer Freundin ins Ausland. Dabei möchten Sie fünf Städte identifizieren, die in Frage kommen, und dabei möglichst verschiedene Optionen betrachten. Die Städte sollen in den Bergen liegen.

1.2 Profil

Die Person legt besonderen Wert auf günstiges Wohnen, preiswertes Essen (selbst gekocht oder unterwegs gekauft), niedrige Telefon- und Internetkosten. Die Person besitzt keinen Führerschein, nutzt öffentliche Verkehrsmittel und möchte zentral in einem belebten Stadtzentrum wohnen.

Neben finanziellen Aspekten ist der Person die Umgebung wichtig: Die Stadt sollte in den Bergen liegen, möglichst über 1000 Höhenmeter.

1.3 Ziel

Es sollen die individuellen Lebenshaltungskosten für die Person ermittelt werden, dazu wird ein Warenkorb für die Person definiert, welcher die Preise im Datensatz sinnvoll gewichtet.

Außerdem sollen Höhendaten genutzt werden, um die Höhenlage sowie die Bergigkeit der Umgebung zu bewerten.

Anhand der Analyse sollen dann Vorschläge für Städte gemacht werden. Die Vorschläge sollen dabei möglichst divers sein. Die Städte sollten:

- auf mindestens 3 unterschiedlichen Kontinenten liegen,
- sowohl Klein- als auch Großstädte umfassen und
- im Ausland liegen.

2. Data Preparation & Feature Engineering

```
In [28]: import pandas as pd
import json

# Lade die CSV-Datei und benenne die Spalten in lesbarer Form um
df = pd.read_csv("data/cost-of-living.csv", index_col=0)
df.rename(columns=json.load(open("data/attribute_names.json")), inplace=True
```

2.1. Filtern der Daten

2.1.1. Filtern anhand von Attributwerten

Zu Beginn werden die Datensätze gefiltert.

Zum Einen können Daten mit explizit schlechter Qualität ausgeschlossen werden, zum Anderen alle deutschen Städte, da die Person explizit *ins Ausland* möchte.

```
In [29]: df = (
    df
    [(df["data_quality"]==1) & (df["country"]!="Germany")]
    .reset_index(drop=True)
    .drop("data_quality", axis="columns")
)
```

2.1.2. Fehlende Werte

Im nächsten Schritt wurde analysiert, welche Merkmale am meisten fehlende Werte aufweisen. Die zehn Merkmale mit den meisten fehlenden Werten wurden visualisiert.

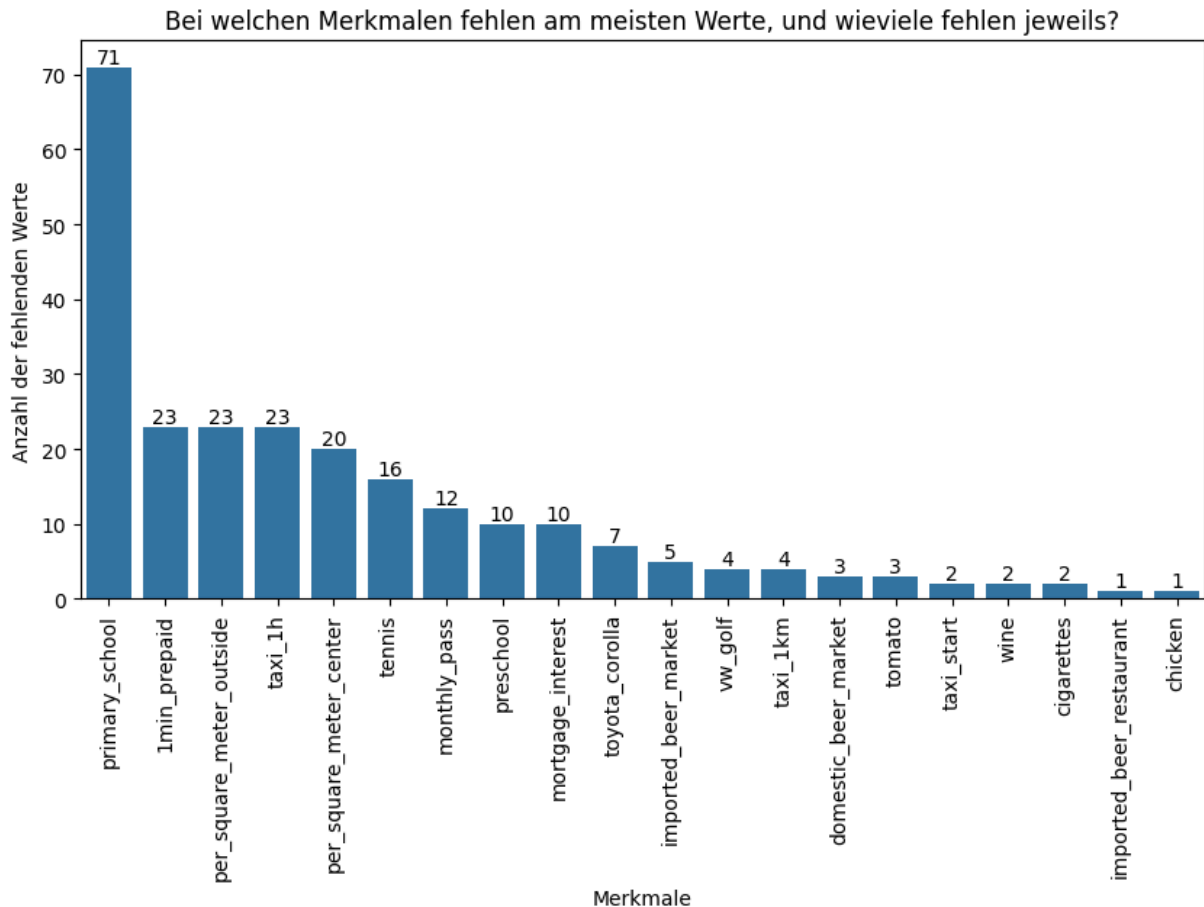
```
In [30]: import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

df_numeric = df.select_dtypes(np.number)

# Die 10 Features ermitteln, bei denen am meisten Werte fehlen
features_with_most_missing = df_numeric.isna().sum().sort_values(ascending=False)

# Darstellen als Seaborn plot
fig, ax = plt.subplots(figsize=(10,5))
ax = sns.barplot(features_with_most_missing)
ax.set(
    title="Bei welchen Merkmalen fehlen am meisten Werte, und wieviele fehle",
    ylabel="Anzahl der fehlenden Werte",
    xlabel="Merkmale",
```

```
)
ax.bar_label(ax.containers[0])
plt.xticks(rotation=90)
plt.show()
```

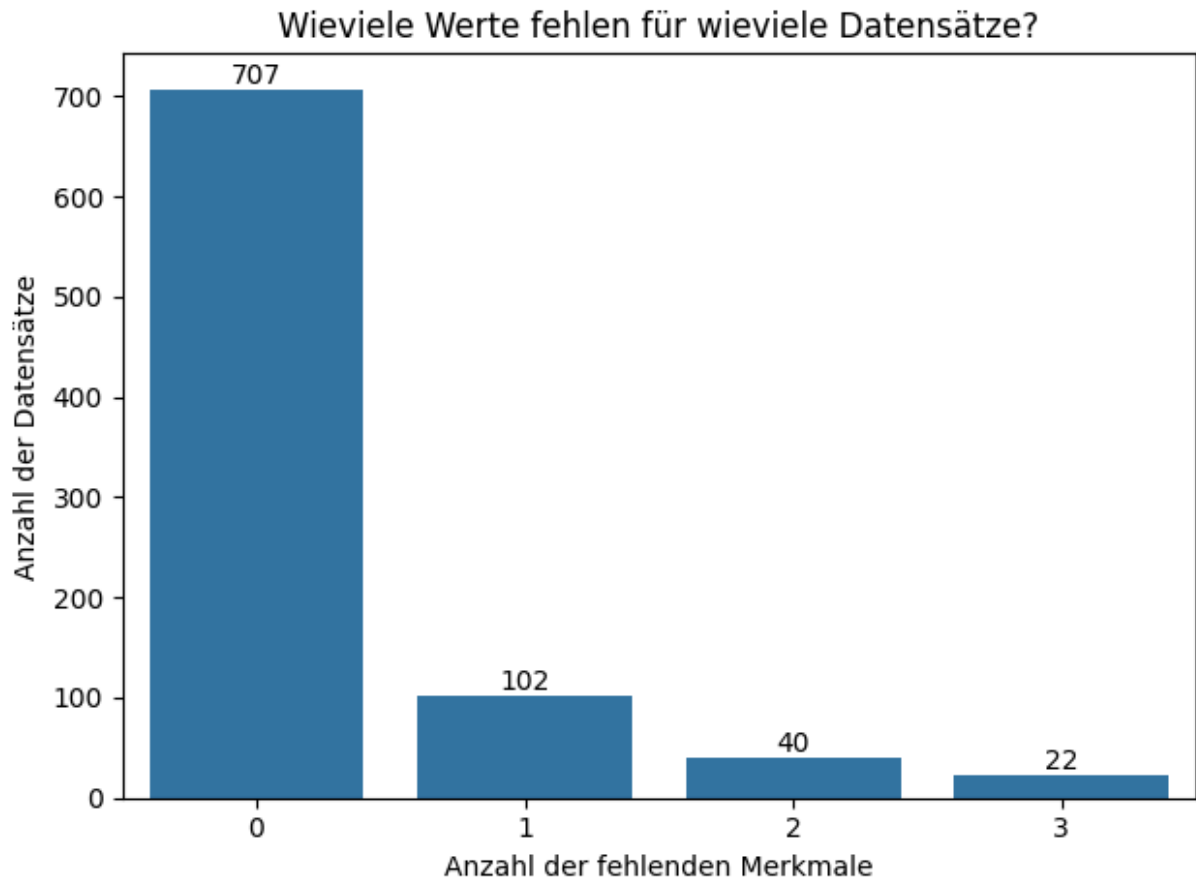


Auf Basis dieser Visualisierung wurde das Merkmal `primary_school` entfernt. Es weist nicht nur am meisten fehlende Werte auf, sondern spielt auch durch die Gewichtung von Null im Warenkorb später keine Rolle.

Desweiteren wurde analysiert, wieviele Werte für wieviele Datensätze fehlen.

```
In [31]: df = df.drop("primary_school", axis="columns")

missing = df_numeric.T.isna().sum().value_counts()
ax = sns.barplot(data=missing)
ax.set(
    title="Wieviele Werte fehlen für wieviele Datensätze?",
    ylabel="Anzahl der Datensätze",
    xlabel="Anzahl der fehlenden Merkmale",
)
ax.bar_label(ax.containers[0])
plt.tight_layout()
plt.show()
```



Diese Visualisierung beweist, dass die Anzahl der fehlenden Werte in den Daten akzeptabel ist. Zum Einen haben die meisten Datensätze gar keine fehlenden Werte, zum Anderen liegt die maximale Anzahl an fehlenden Werten bei 3, was im Vergleich zur Merkmalsanzahl sehr klein ist.

Für folgende Berechnungen ist es trotzdem notwendig, dass keine Werte fehlen. Deswegen wurden die fehlenden Werte durch den Median des jeweiligen Attributes ersetzt. Dies ändert den Datensatz zwar statistisch minimal, erhält aber die anderen, vorhandenen Merkmalsinformationen der betroffenen Datensätze. Somit ist es besser als das Dropen aller Datensätze, denen mindestens ein Wert fehlt.

```
In [32]: numeric_columns = df.dtypes.index[df.dtypes=="float64"]
df_numeric = df[numeric_columns]
print("Filling " + str(df_numeric.isna().sum().sum()) + " na values.")
df.loc[:, numeric_columns] = df_numeric.fillna(df_numeric.median())
```

Filling 177 na values.

2.1.3. Ausreißer

Zur Ausreißererkennung haben wir die Filterung anhand der Mahalanobi-distanz verwendet. Sie stellt (schwer vereinfacht) eine Extremwertanalyse für Daten mit vielen Dimensionen dar, die darüber hinaus auch noch Korrelationen zwischen diesen berücksichtigt.

Dieses Verfahren bietet darüber hinaus noch die folgenden Vorteile (Aggarwal C.C., 2017, S.53):

- Es ist *nicht parametrisiert* -> es muss keine richtige Wahl für Konfigurationsparameter getroffen werden
- Es ist *recheneffizient* ($O(n)$ zur Anzahl der Datensätze)

Zuallererst wird ein Mittelpunkt des Datensatzes und die Inverse der Kovarianzmatrix der Daten berechnet. Mit diesen kann dann für jeden Datensatz eine Distanz berechnet werden.

Anhand dieser Distanz wird der Datensatz dann gefiltert. Die Strenge für diese Filterung wird vom Parameter `a` bestimmt, der angibt, wieviel Prozent des Datensatzes ausgefiltert werden.

Schlussendlich visualisiert das Balkendiagramm noch die Verteilung der Distanz. Die rote Linie stellt hierbei die Filtergrenze dar.

```
In [33]: # from scipy.spatial.distance import mahalanobis

# df_numeric = df.select_dtypes(np.number)

# # Mittelpunkt berechnen
# middle = (df_numeric.sum()/df_numeric.count()).to_numpy()

# # Inverse der Kovarianzmatrix der Daten berechnen (für Berechnung der Maha
# inv_cov = np.linalg.inv(np.cov(df_numeric.T))

# # Berechnung der Mahalanobisdistanz für alle Städte
# mahalanobis_series = df_numeric.apply(lambda x: mahalanobis(x, middle, inv

# # Wert für Filter mit Quantil berechnen
# a = .01
# quantile = mahalanobis_series.quantile(1-a)

# # Löschen der Datensätze
# print(str(df_numeric[mahalanobis_series > quantile].shape[0]) + \
#       " Ausreißer werden entfernt.")
# df = df[mahalanobis_series <= quantile]

# # Mahalanobi Distanzen visualisieren
# ax = sns.histplot(mahalanobis_series, bins=40)
# ax.set(
#     title=f"Verteilung der Mahalanobi-distanzen, Filtergrenze = {quantile:
#     xlabel="Mahalanobi-distanz",
#     ylabel="Anzahl der Datensätze",
# )
# plt.axvline(x=quantile, color="r", label = "Filtergrenze")
# plt.show()
```

2.1.4. Duplikate

Im nächsten Schritt wird der Datensatz auf Duplikate untersucht. Da keine Duplikate vorliegen, muss deswegen auch kein Datensatz entfernt werden.

```
In [34]: city_duplicates = df["city"].value_counts(>1) # doppelte Städtenamen
# doppelte Städtenamen die nicht in verschiedenen Ländern vorkommen
city_country_duplicates = df[["city", "country"]].value_counts(>1)

print(f"Es gibt zwar {city_duplicates.sum()} doppelte Städtenamen, \
jedoch {city_country_duplicates.sum()} doppelte Städtenamen in verschiedenen
```

Es gibt zwar 8 doppelte Städtenamen, jedoch 0 doppelte Städtenamen in verschiedenen Ländern.

2.2. Anreichern der Daten

Die folgenden Zellen stellen das Anreichern des Dataframes um die folgenden Daten:

- Länderkürzel
- Kontinentnamen
- Geographische Koordinaten
- Population
- Höhenmeter

2.2.1. Länderkürzel & Kontinentnamen

Der Kontinent der einzelnen Städte wird mithilfe der Python-bibliothek `pycountry-convert` ermittelt. Hierbei werden zuerst die country codes ermittelt und abgespeichert, aus denen über die continent codes die Namen der Kontinente ermittelt werden können.

Einige wenige Ausnahmen, die nicht von der Bibliothek abgehandelt werden können, wurden manuell recherchiert und über die Dictionaries `exceptions` abgehandelt.

```
In [35]: import pycountry_convert as pc

# Umwandlung der Ländernamen in Länderkürzel
def country_name_to_alpha2(country_name):

    # Ausnahmefall
    exceptions = {
        "Bosnia And Herzegovina": "BA",
        "Trinidad And Tobago": "TT",
        "Isle Of Man": "IM",
        "Curacao": "CW",
        "Kosovo (Disputed Territory)": "XK",
    }
    if country_name in exceptions.keys():
        return exceptions[country_name]
```

```

# Regelfall
country_alpha2 = pc.country_name_to_country_alpha2(country_name)
return country_alpha2

df["country_code"] = df["country"].apply(country_name_to_alpha2)

```

```

In [36]: # Umwandlung der Länderkürzel in Kontinentkürzel und -namen
def alpha2_to_continent(alpha2):

    # Ausnahmefall
    exceptions = {
        "BA": "Europe",
        "TT": "South America",
        "IM": "Europe",
        "CW": "South America",
        "XK": "Europe",
    }
    if alpha2 in exceptions.keys():
        return exceptions[alpha2]

    # Regelfall
    continent_code = pc.country_alpha2_to_continent_code(alpha2)
    continent_name = pc.convert_continent_code_to_continent_name(continent_code)
    return continent_name

df["continent"] = df["country_code"].apply(alpha2_to_continent)

```

2.2.2. Geographische Koordinaten und Population

Der von [GeoNames](#) stammende Datensatz umfasst Städte mit mindestens 15000 Einwohnern oder Hauptstädte. Über ihn werden geographische Koordinaten und Populationsdaten angereichert. Im ersten Schritt wird er geladen und die relevanten Zeilen mithilfe des Arguments `usecols=` ausgewählt.

Um für eine Stadt die entsprechenden Daten in `cities` abzurufen, wird zuerst ein direkter Match mittels Städtename und Länderkürzel versucht. Scheitert dieser, wird überprüft, ob der Name der Stadt aus `df` als ein alternativer Name einer Stadt aus `cities` auftaucht. Alle Städte, die auch so nicht bearbeitet werden können, werden analog zum oberen Vorgehen mit `exceptions` abgehandelt.

```

In [37]: # Laden der Daten als Pandas DataFrame; Auswahl relevanter Spalten
cities = pd.read_table(
    "data/cities15000.txt",
    names=[
        "geonameid", "name", "asciiname", "alternatenames", "latitude", "lon
        "feature class", "feature code", "country code", "cc2", "admin1 code
        "admin3 code", "admin4 code", "population", "elevation", "dem", "tim
    ],
    usecols=["asciiname", "country code", "latitude", "longitude", "populati
)
# Vorverarbeitung: Eindeutige Stadt<->Staat Paare
cities = cities.groupby(["asciiname", "country code"]).first().reset_index()

```

```

# Vorverarbeitung: Umwandlung der alternatenames-Information von String zu List
cities["alternatenames"] = cities["alternatenames"].apply(lambda x: x.split(

exceptions = {
    "Windhoek": (-22.57, 17.083611, 486169),
    "Bhubaneshwar": (20.27, 85.84, 843402),
    "Sialkot City": (32.497222, 74.536111, 655852),
    "`Ajman": (25.4, 55.433333, 489176),
    "Kocaeli": (40.763889, 29.941667, 1997258),
    "Sandton": (-26.106944, 28.051667, 222415),
    "Jyvaskyla": (62.233056, 25.733056, 145887),
    "Targu-Mures": (46.533333, 24.566667, 116033),
    "Vasteras": (59.611111, 16.545833, 117746),
    "Gavle": (60.675556, 17.145833, 74884),
    "Southwick": (50.836, -0.239, 13195),
    "Gzira": (35.9049, 14.4938, 13021),
    "Joinville": (-26.301389, -48.843889, 515288),
    "Ra'ananna": (32.183333, 34.866667, 73999),
    "Walton upon Thames": (51.386797, -0.413394, 22834),
}

def fill_data(city_row):
    city_name = city_row["city"]
    country_code = city_row["country_code"]

    # Direkten Match mit Städtename und Länderkürzel suchen
    city_name_matches = cities["asciiiname"]==city_name
    country_code_matches = cities["country code"]==country_code
    direct_matches = cities[city_name_matches & country_code_matches]
    direct_match_exists = len(direct_matches)>0

    if direct_match_exists:
        direct_match = direct_matches.iloc[0]
        city_row["latitude"] = direct_match["latitude"]
        city_row["longitude"] = direct_match["longitude"]
        city_row["population"] = direct_match["population"]
        return city_row

    # Match über alternative Städtenamen ermitteln
    alt_names_matches_index = cities["alternatenames"]\
        .apply(lambda alt_names_list: city_name in alt_names_list)
    alt_names_matches = cities[alt_names_matches_index]
    alt_names_match_exists = len(alt_names_matches)>0

    if alt_names_match_exists:
        first_alt_match = alt_names_matches.iloc[0]
        city_row["latitude"] = first_alt_match["latitude"]
        city_row["longitude"] = first_alt_match["longitude"]
        city_row["population"] = first_alt_match["population"]
        return city_row

    # Ausnahmen abhandeln, die nicht in Datensatz gefunden werden
    if city_name in exceptions.keys():
        exception_data = exceptions[city_name]
        city_row["latitude"] = exception_data[0]

```



```

city_row["longitude"] = exception_data[1]
city_row["population"] = exception_data[2]
return city_row

```

```
df = df.apply(fill_data, axis="columns")
```

2.2.3. Höhenmeter

Der *Global Multiresolution Terrain Elevation Data*-Datensatz (GMTED2010) stellt ein globales Höhenprofil als Raster dar. (Dimensionen 1440*560)

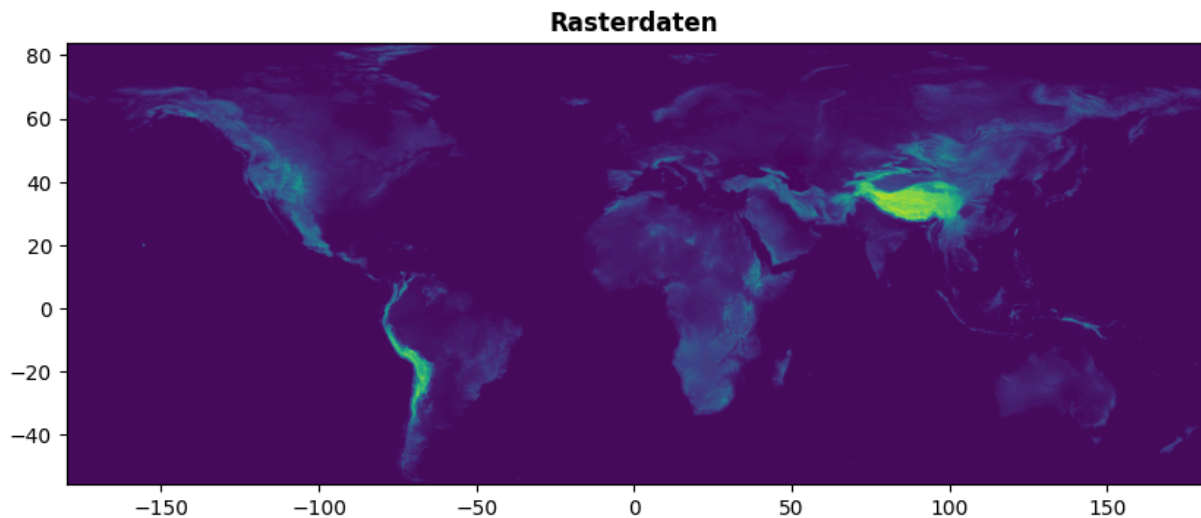
Über diesen können nicht nur Höhendaten für einzelne Geokoordinaten abgefragt werden.

```

In [38]: import rasterio
from rasterio.plot import show

with rasterio.open("data/GMTED2010_averaged.tif") as src:
    fig, ax = plt.subplots(figsize=(10, 10))
    show(src, ax=ax, title="Rasterdaten")
    plt.show()

```



Die Funktion `get_height_at_coordinates` erlaubt die Umrechnung von Geokoordinaten auf Höhenmeter

```

In [39]: from rasterio.transform import rowcol

def get_height_at_coordinates(longitude, latitude, raster_file="data/GMTED20
    """
    Gibt die Höhe für gegebene geografische Koordinaten (Longitude, Latitude

    :param longitude: Longitude-Wert (Längengrad)
    :param latitude: Latitude-Wert (Breitengrad)
    :param raster_file: Pfad zur Rasterdatei
    :return: Höhe an den angegebenen Koordinaten
    """

```

```

with rasterio.open(raster_file) as src:
    # Transformiere die geografischen Koordinaten in Raster-Koordinaten
    row, col = rowcol(src.transform, longitude, latitude)

    # Hole die Höhendaten als numpy array
    height_data = src.read(1) # Liest das erste Band (Höhendaten)

    # Hole den Wert aus den Rasterdaten
    value = height_data[row, col]
    return value

# Beispielnutzung der Funktion
latitude, longitude = 48.782624819735624, 9.167036756166327 # Stuttgart
höhe = get_height_at_coordinates(longitude, latitude) # Der Wert ist die du
print(f"Höhe an der Position ({longitude}, {latitude}): {höhe}")

```

Höhe an der Position (9.167036756166327, 48.782624819735624): 294

```

In [40]: df["elevation"] = df.apply(
    lambda row: get_height_at_coordinates(row["longitude"], row["latitude"])
    axis=1
)

```

3. Data Exploration und Analyse

Viele der Features im Datensatz beziehen sich auf Preise. Die Wichtigkeit eines einzelnen Preismerkmals ist jedoch von Person zu Person unterschiedlich. Um die Kauf- und Lebensweise, die in Kapitel 1 spezifiziert wurde, abzubilden wurde mittels eines "Warenkorbs" eine Gewichtung für die einzelnen Merkmale vorgenommen. Diese stellt ein auf die Person angepasstes Preis-rating der einzelnen Städte dar.

Um eine generelle Analyse der Kosten durchführen zu können werden die Kosten dann noch in Kategorien eingeteilt.

```

In [41]: # Copy des DataFrames für die Analyse
df_analyze = df.copy()

# JSON-Daten laden
with open("data/categories.json", "r") as file:
    weights = json.load(file)

# Funktion zur Berechnung der gewichteten Werte für jede Kategorie
def calculate_weighted_values(df_analyze, weights):
    weighted_values = {}
    for category, items in weights.items():
        weighted_values[category] = df_analyze[list(items.keys())].mul(pd.Series(
    return weighted_values

# Gewichtete Werte berechnen
weighted_values = calculate_weighted_values(df_analyze, weights)

# Gewichtete Werte als neue Spalten hinzufügen

```

```

for category, values in weighted_values.items():
    df_analyze[f"weighted_{category}"] = values

# Gruppierte Balkendiagramm für gewichtete Attribute nach Kontinent
weighted_columns = [col for col in df_analyze.columns if col.startswith("wei
df_analyze["weighted_basked"] = df_analyze[weighted_columns].sum(axis=1)

```

3.1 Auswertung nach Kontinenten

Generelle Verteilung

```

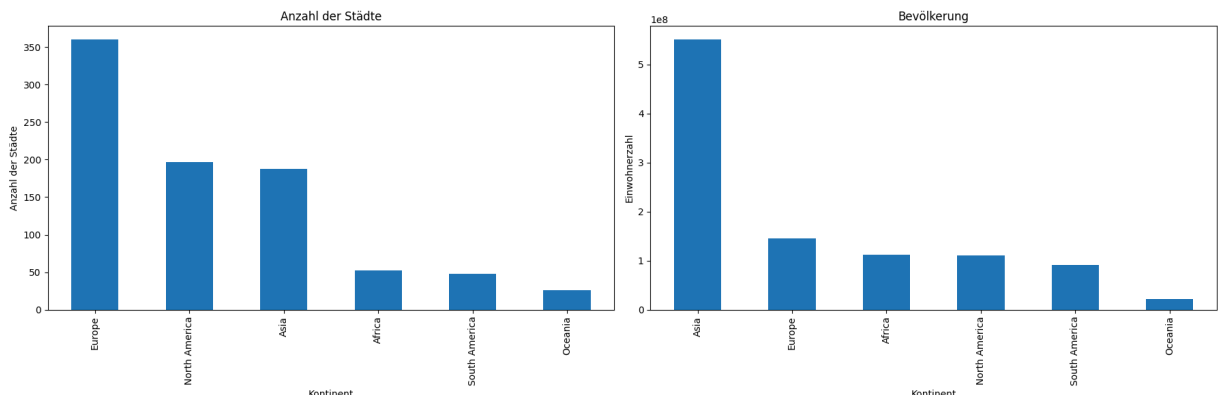
In [42]: fig, axes = plt.subplots(1, 2, figsize=(18, 6), sharey=False)

# Diagramm 1: Bevölkerung nach Kontinent
df_analyze.groupby("continent")["population"].sum().sort_values(ascending=False,
    kind="bar", ax=axes[1], title="Bevölkerung", ylabel="Einwohnerzahl", xla
)

# Diagramm 2: Zahl der Städte nach Kontinent
df_analyze.groupby("continent")["city"].count().sort_values(ascending=False)
    kind="bar", ax=axes[0], title="Anzahl der Städte", ylabel="Anzahl der St
)

plt.tight_layout()
plt.show()

```



Ausgabenstruktur

Die folgende Grafik zeigt, dass sich die monatlichen Lebenshaltungskosten des definierten Warenkorbs zwischen den Kontinenten deutlich unterscheiden. Der prozentuale Anteil der einzelnen Kategorien (z. B. Essen, Wohnen etc.) an den Gesamtkosten bleibt jedoch weitgehend ähnlich.

```

In [43]: import pandas as pd
import json

def plot_expenses(df_analyze, groub_by_col, title):

```

```

# Erstellen eines gestapelten Balkendiagramms für jedes Kontinent
grouped_data = df_analyze.groupby(groub_by_col)[weighted_columns].mean()

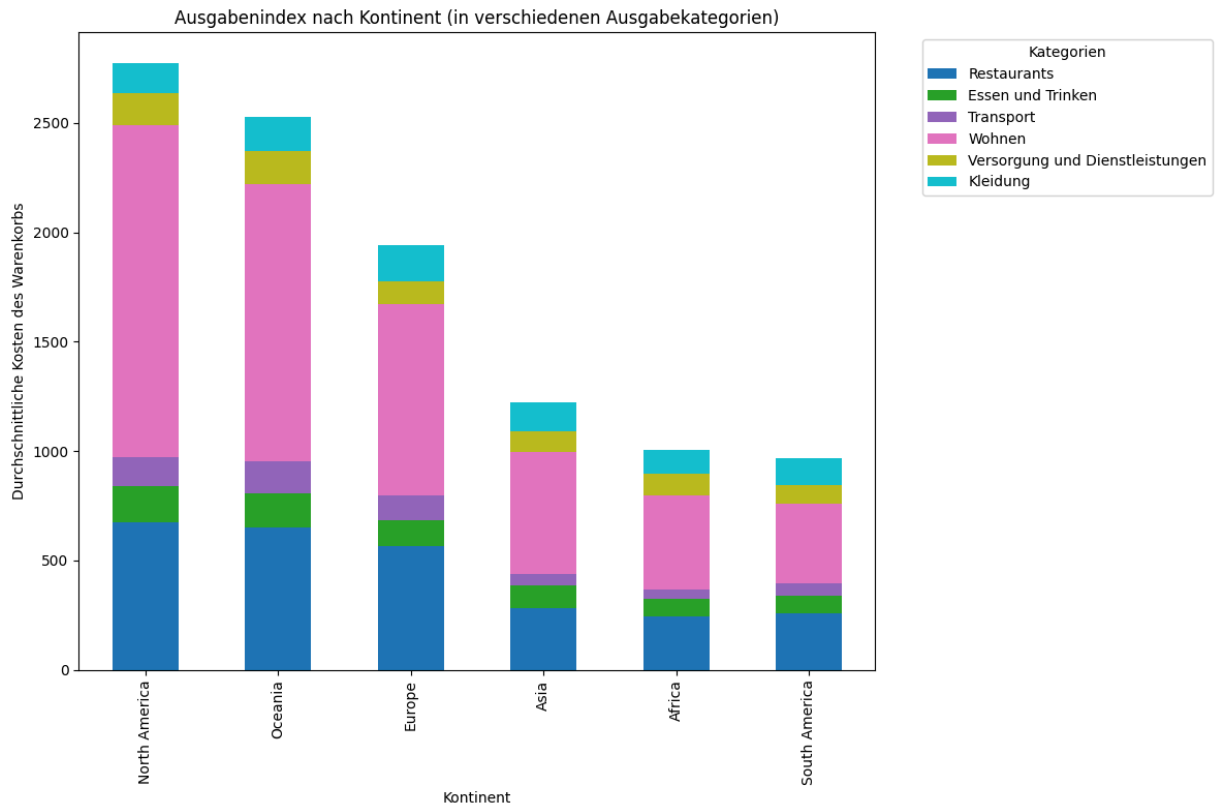
# Sortiere die Daten nach der größten Gesamtausgabe
grouped_data["sum"] = grouped_data.sum(axis=1)
grouped_data = grouped_data.sort_values(by="sum", ascending=False)

# Gestapeltes Balkendiagramm
grouped_data.drop(columns=["sum"], inplace=True)
grouped_data.rename(columns={
    "weighted_restaurants": "Restaurants",
    "weighted_food_and_drinks": "Essen und Trinken",
    "weighted_transportation": "Transport",
    "weighted_housing": "Wohnen",
    "weighted_utilities_and_services": "Versorgung und Dienstleistungen",
    "weighted_clothing": "Kleidung"
}, inplace=True)

grouped_data.plot(
    kind="bar",
    stacked=True,
    figsize=(12, 8),
    title=title,
    ylabel="Durchschnittliche Kosten des Warenkorb",
    xlabel="Kontinent",
    colormap="tab10",
)
#plt.xticks(rotation=45)
plt.legend(title="Kategorien", bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()

plot_expenses(df_analyze, "continent", "Ausgabenindex nach Kontinent (in ver

```

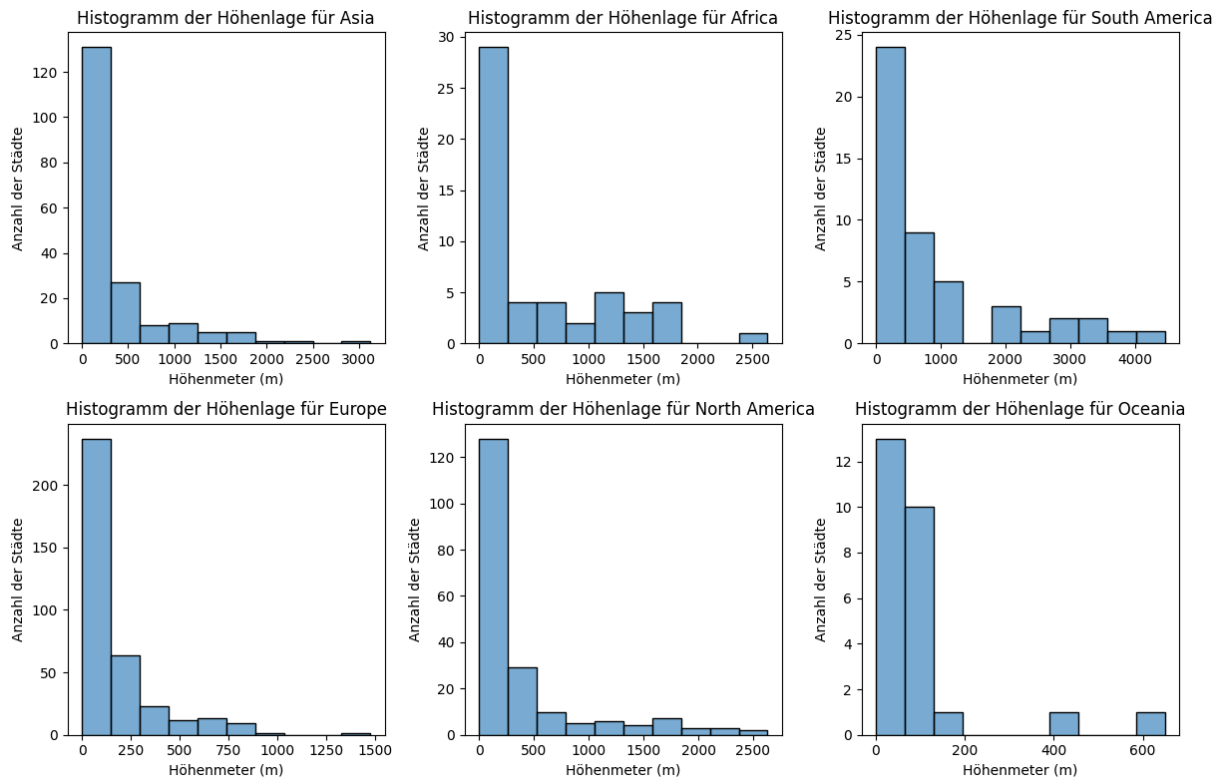


```
In [44]: continents = df_analyze["continent"].unique()
fig, axes = plt.subplots(2, 3, figsize=(12, 8))

for ax, continent in zip(axes.flat, continents):
    group = df_analyze[df_analyze["continent"] == continent]
    sns.histplot(
        group["elevation"],
        kde=False,
        bins=10,
        alpha=0.6,
        ax=ax
    )
    ax.set_title(f"Histogramm der Höhenlage für {continent}")
    ax.set_xlabel("Höhenmeter (m)")
    ax.set_ylabel("Anzahl der Städte")

# Entferne leere Subplots, falls weniger als 6 Kontinente vorhanden sind
for ax in axes.flat[len(continents):]:
    ax.axis("off")

plt.tight_layout()
plt.show()
```



3.1 Auswertung nach Ländern

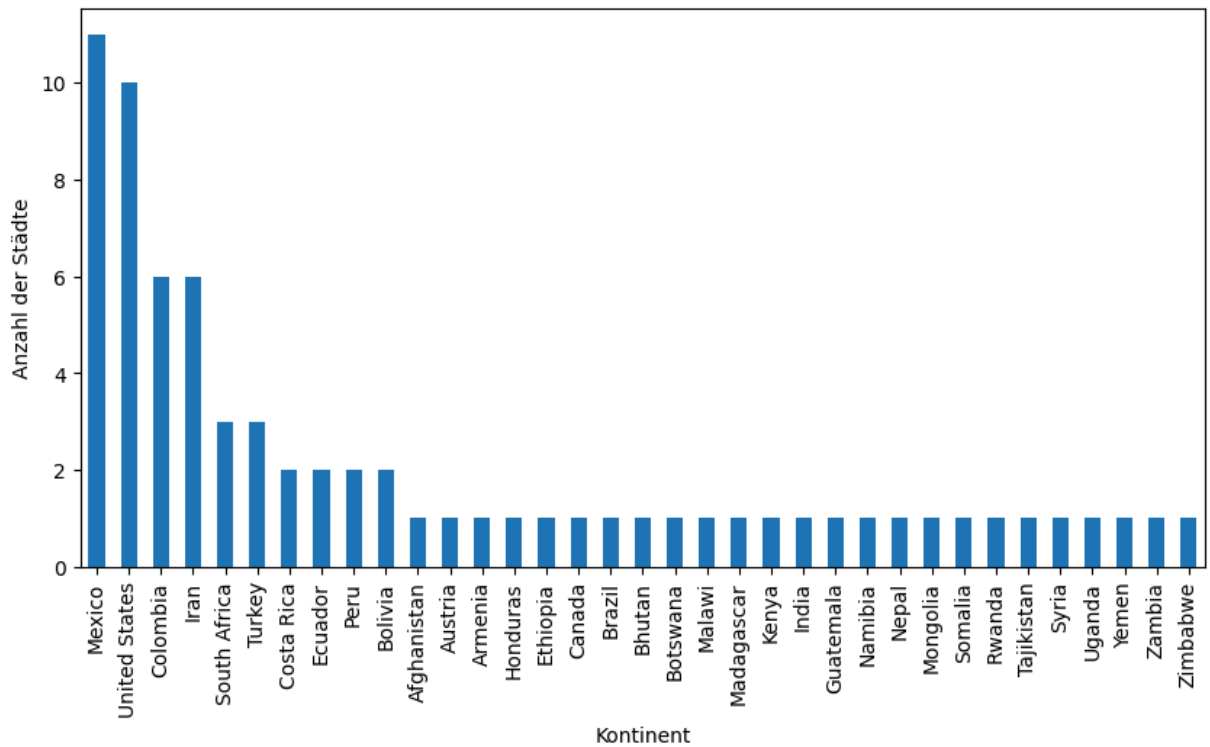
Vorab wird die Restriktion auf Städte die höher als 1000 Höhenmeter liegen angewendet, dies ist eine Beschränkung aus der Aufgabenstellung. Es macht die Auswertung übersichtlicher, sich bereits hier darauf zu beschränken.

```
In [45]: df_analyze = df_analyze[df_analyze["elevation"] > 1000]

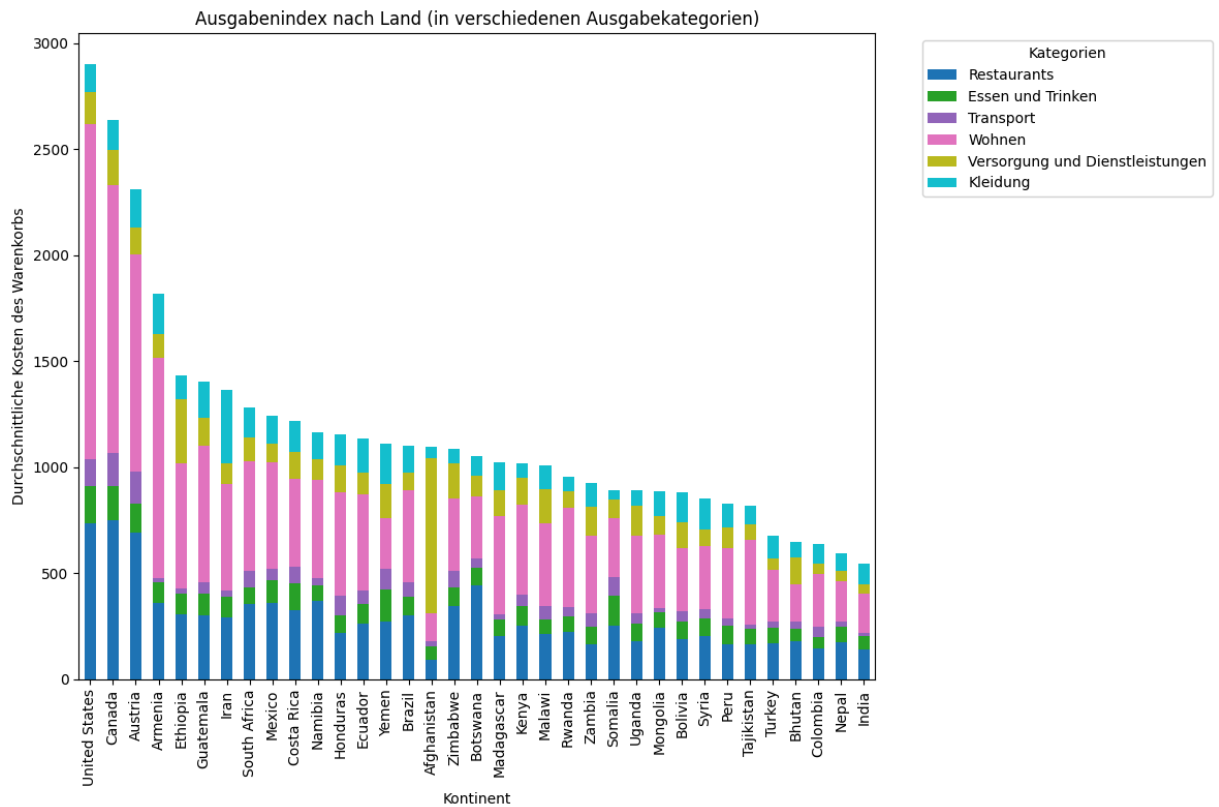
df_analyze.groupby("country")["city"].count().sort_values(ascending=False).p
    kind="bar",
    title="Anzahl der Städte über 1000m Höhe nach Land",
    ylabel="Anzahl der Städte",
    xlabel="Kontinent",
    figsize=(10, 5)
)
```

```
Out[45]: <Axes: title={'center': 'Anzahl der Städte über 1000m Höhe nach Land'}, xla
    bel='Kontinent', ylabel='Anzahl der Städte'>
```

Anzahl der Städte über 1000m Höhe nach Land



In [46]: `plot_expenses(df_analyze, "country", "Ausgabenindex nach Land (in verschiedene`



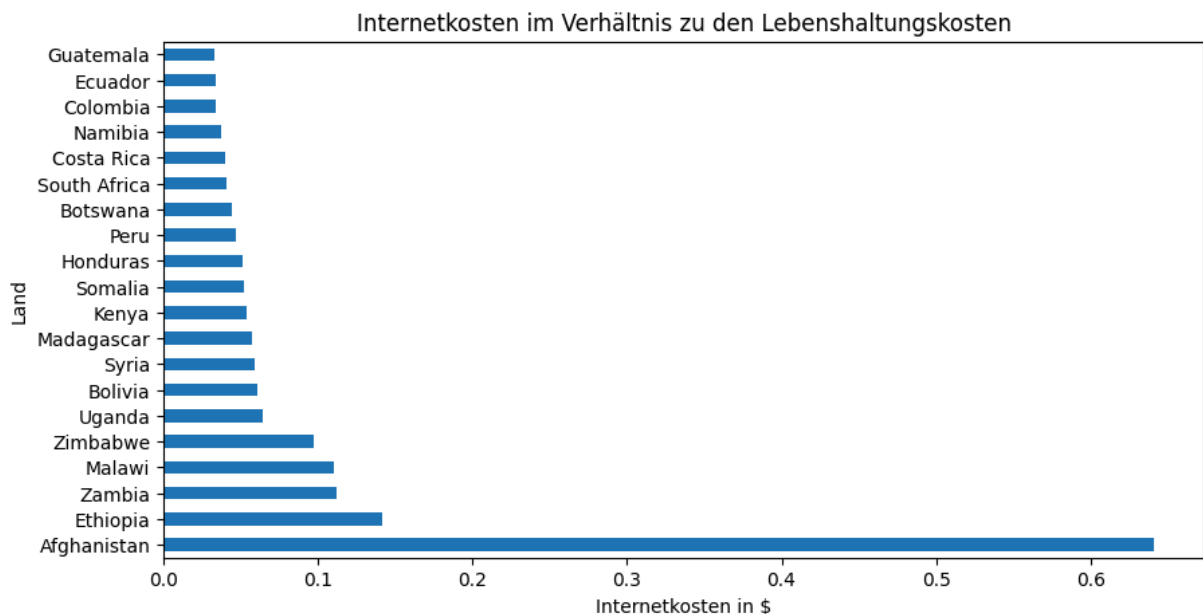
Betrachtung Internetkosten

Generell ist für die Person eine Stabile internetverbindung wichtig (Remote Worker), eine analyse der Merkmale hatte ergeben das in einigen weniger stark entwickelten ländern Internetkosten sehr hoch sind. Da dies nicht mit dem generellen Preisniveau der länder übereinstimmt. Ist dies ein Hinweis auf die schlechte internetverfügbarkeit.

```
In [47]: df_analyze["internet_coefitiant"] = df_analyze["internet"] / df_analyze["we

highest_internet_costs = df_analyze.groupby("country")["internet_coefitiant"]
highest_internet_costs.head(20).plot(
    kind="barh",
    title="Internetkosten im Verhältnis zu den Lebenshaltungskosten",
    ylabel="Land",
    xlabel="Internetkosten in $",
    figsize=(10, 5)
)
```

```
Out[47]: <Axes: title={'center': 'Internetkosten im Verhältnis zu den Lebenshaltungs
kosten'}, xlabel='Internetkosten in $', ylabel='Land'>
```



3.2 Auswahl der Städte

Länder wo die internetkosten mehr als 10% der Lebenshaltungskosten ausmachen werden deshalb aussortiert, da hier die Gefahr für eine schlechte internetverbindung als zu hoch angesehen wird.

```
In [48]: df_analyze = df_analyze[df_analyze["internet_coefitiant"] < 0.1]
```

```
In [49]: df_analyze = df_analyze[df_analyze["weighted_basked"] < 3000]
```

```
In [50]: import kaleido
import plotly.express as px
from IPython.display import Image
```



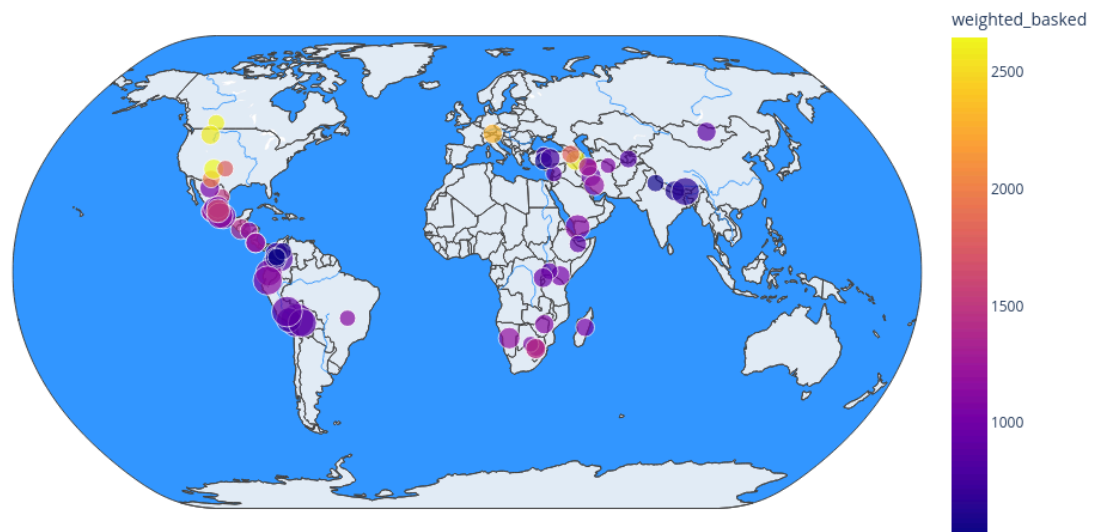
```

fig = px.scatter_geo(
    df_analyze,
    lat="latitude",
    lon="longitude",
    size="elevation",
    color="weighted_basked",
    hover_name="city",
    hover_data=["country", "population", "elevation", "weighted_basked"],
    projection="natural earth",
    height=600,
    width=1000,
)
fig.update_geos(
    resolution=110,
    showcountries=True,
    showlakes=True,
    showland=True,
    showocean=True,
    showrivers=True,
    showcoastlines=True,
)
fig.update_layout(
    title=f"Countries plotted by geo coordinates",

    showlegend=False,
)
Image(fig.to_image())

```

Out[50]: Countries plotted by geo coordinates



Um die Vorschläge möglichst divers zu machen wird für jedes Kontinent ein Vorschlag gemacht und zwar wird immer das land mit dem geringsten Kostenindex ausgewählt.

```
In [51]: df_analyze.loc[df_analyze.groupby("continent")["weighted_basked"].idxmin()]
```

```
Out[51]:
```

	city	country	continent	weighted_basked	elevation
174	Kampala	Uganda	Africa	891.33375	1163
667	Konya	Turkey	Asia	533.70250	1302
709	Innsbruck	Austria	Europe	2311.37000	1476
324	Aguascalientes	Mexico	North America	914.93000	1948
446	Manizales	Colombia	South America	518.70375	1264

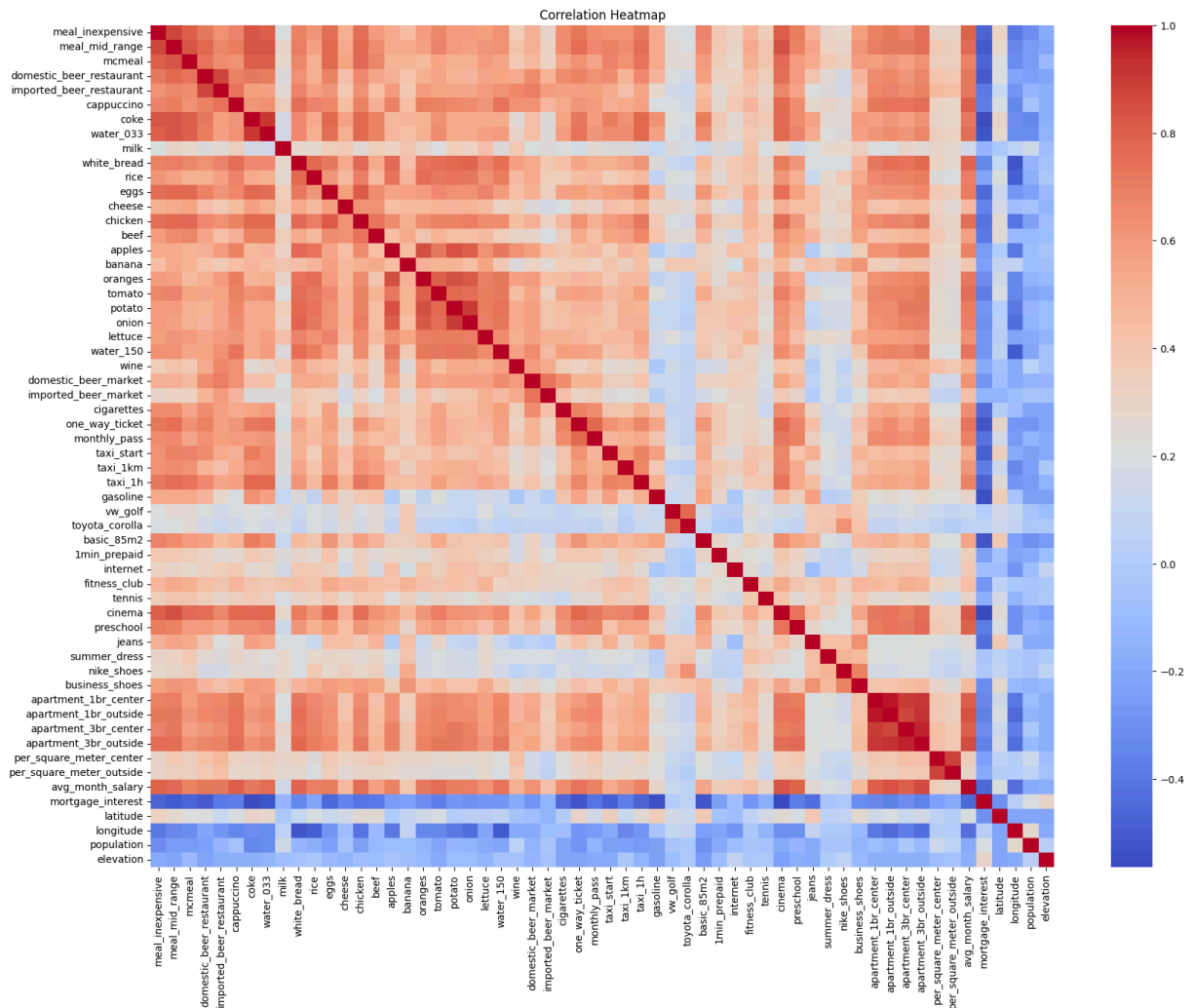
Korrelationsmatrix

```
In [52]: import matplotlib.pyplot as plt
import seaborn as sns

numeric_columns = df.select_dtypes(np.number)

# Compute correlation matrix
correlation_matrix = numeric_columns.corr()

# Plot heatmap with all column names displayed
plt.figure(figsize=(20, 15))
sns.heatmap(correlation_matrix, annot=False, cmap="coolwarm", fmt=".2f", xti
plt.title("Correlation Heatmap")
plt.xticks(rotation=90)
plt.yticks(rotation=0)
plt.show()
```



4. Modelling und Evaluation Regression

Der Applepreis wird anhand vorhandener Features im Datensatz vorhergesagt. Dazu wird zunächst eine Feature-Selektion durchgeführt, und anschließend ein Estimator verwendet, um den Preis zu prognostizieren. Um einen Überblick zu gewinnen, werden verschiedene Selektionsverfahren, Estimatoren sowie unterschiedliche Anzahlen an Features getestet. Jedes Ergebnis (R^2 und RMSE) wird im Rahmen einer 5-fachen Kreuzvalidierung ermittelt.

Wichtig: Wenn die Feature-Auswahl beispielsweise auf der Korrelation zwischen dem Applepreis und den anderen Features basiert, werden für die Korrelationsberechnung ausschließlich die Trainingsdaten verwendet. Alles andere würde zu einer Vermischung von Trainings- und Testdaten führen – denn auch die Feature-Auswahl stellt eine Trainingsentscheidung dar. Im Code wird dies dadurch sichergestellt, dass die gesamte Pipeline aus Selektion und Estimator in die Kreuzvalidierung eingebunden wird – und nicht nur der Estimator allein.

```

In [53]: from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.svm import SVR
from sklearn.feature_selection import SelectKBest, f_regression, mutual_info
from sklearn.pipeline import Pipeline

n_values = list(range(1, 20)) # Verschiedene Werte von n, über die iteriert

# Initialisiere Speicher für Ergebnisse
results = []

# Bewertungsfunktion für die Auswahl von Features anhand der Korrelation mit
def compute_correlation(X, y):
    """Berechnet die Korrelation zwischen Features und Zielwert.
    Argumente:
        X: ndarray, Form (n_samples, n_features)
        y: Zielvariable
    Rückgabe:
        ndarray, Form (n_features,)
        Korrelationskoeffizienten zwischen jedem Feature und der Zielvar
    """
    # Numpy-Trick: Die Funktion gibt eigentlich eine Matrix zurück, die letz
    # enthält die gewünschten Koeffizienten
    return np.abs(np.corrcoef(X.T, y)[-1, :-1])

x = numeric_columns.drop(columns=["apples"]).dropna()
y = numeric_columns["apples"].loc[x.index]

for n in n_values:

    # Definiere Methoden zur Feature-Auswahl
    feature_selection_methods = {
        "f_regression": SelectKBest(score_func=f_regression, k=n),
        "mutual_info": SelectKBest(score_func=mutual_info_regression, k=n),
        "correlation": SelectKBest(score_func=compute_correlation, k=n),
    }

    # Definiere Modelle
    base_estimators = {
        "Lineare Regression": LinearRegression(),
        "SVR": SVR(kernel="rbf"),
        "Random Forest": RandomForestRegressor(n_estimators=100),
        "K-Nearest Neighbors": KNeighborsRegressor(n_neighbors=5),
    }

    # Erstelle Pipelines für alle Kombinationen aus Feature-Auswahl und Mode
    estimators = {
        (fs_name, est_name): Pipeline([
            ("feature_selection", fs_method),
            ("estimator", estimator)
        ])
        for fs_name, fs_method in feature_selection_methods.items()
    }

```

```

        for est_name, estimator in base_estimators.items()
    }

    # Trainiere und bewerte jedes Modell
    for name, estimator in estimators.items():
        scores_r2 = cross_val_score(estimator, x, y, cv=5, scoring="r2")
        scores_rmse = np.sqrt(-cross_val_score(estimator, x, y, cv=5, scorin

        results.append({
            "n": n,
            "Feature-Auswahl": name[0],
            "Modell": name[1],
            "RMSE": scores_rmse.mean(),
            "R2": scores_r2.mean(),
            "RMSE Std": scores_rmse.std(),
            "R2 Std": scores_r2.std()
        })

    # Ergebnisse mit MultiIndex anzeigen
    results_df = pd.DataFrame(results).set_index(["Modell", "Feature-Auswahl"])

```

Hinweis: Vor allem bei der Selektion zeigten sich univariate Verfahren wie beispielsweise Forward- oder Backward-Selektion als wenig vielversprechend, weswegen diese in unserem Vergleich nicht weiter berücksichtigt wurden. Für eine Auswertung anderer Estimatoren und Selektionsverfahren muss lediglich in den Dictionaries `feature_selection_methods` oder `base_estimators` eine Zeile ergänzt und ein entsprechendes `sklearn`-Objekt übergeben werden.

4.1 Ergebnisse

Die folgenden Plots zeigen den Verlauf der Metriken für die einzelnen Kombinationen aus Estimator und Selektionsverfahren. Kombinationen, die eine sehr schwache Performance gezeigt haben – darunter die Ansätze mit SVR und K-Nearest Neighbors – wurden aus Gründen der Übersichtlichkeit entfernt.

```

In [54]: # Erstelle eine Liste der Metriken
metrics = ["RMSE", "R2"]

# Erstelle eine Figure mit Subplots
fig, axes = plt.subplots(1, len(metrics), figsize=(15, 5), sharex=True)

# Iteriere über die Metriken und erstelle für jede einen separaten Plot
for i, metric in enumerate(metrics):
    ax = axes[i]
    for (model, feature_selection), subset in results_df.groupby(["Modell",

        if model in ["SVR", "K-Nearest Neighbors"]:
            continue
        if feature_selection in ["f_regression"]:
            continue

```

```

ax.plot(
    subset["n"],
    subset[metric],
    marker="o",
    label=f"{model} - {feature_selection}"
)
ax.set_title(f"Verlauf der {metric} in Abhängigkeit von n")
ax.set_xlabel("n")
ax.set_ylabel(metric)
ax.legend(title="Model - Feature Selection", bbox_to_anchor=(1.05, 1), 1
ax.grid(True)

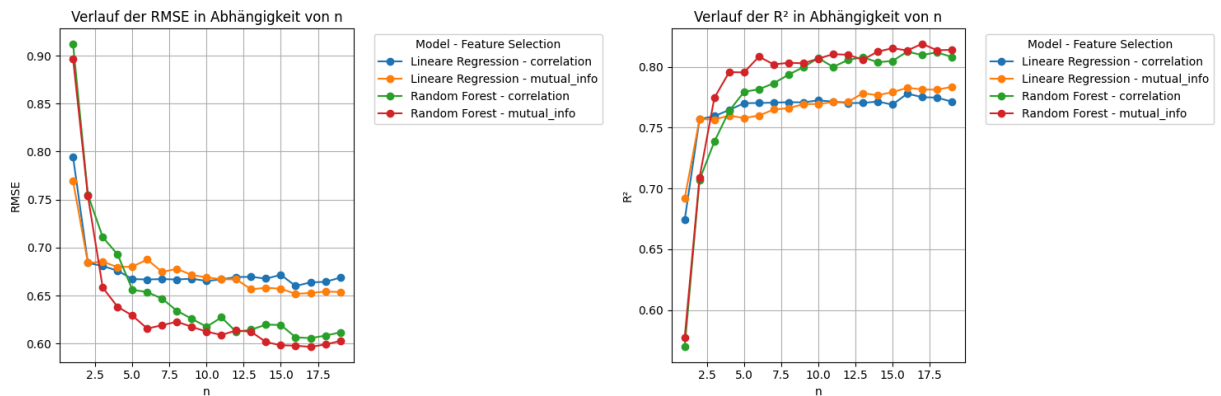
```

Passe das Layout an

```

plt.tight_layout()
plt.show()

```



Varianzbetrachtung

```

In [55]: results_df[results_df["n"] == 6].groupby(["Modell", "Feature-Auswahl"]).mean

```

Out[55]:

		n	RMSE	R ²	RMSE Std	R ² Std
Modell	Feature-Auswahl					
Random Forest	mutual_info	6.0	0.615547	0.808371	0.107811	0.050283
K-Nearest Neighbors	correlation	6.0	0.644584	0.787249	0.106149	0.046189
	f_regression	6.0	0.644584	0.787249	0.106149	0.046189
Random Forest	f_regression	6.0	0.650015	0.783563	0.112618	0.049338
	correlation	6.0	0.653604	0.781555	0.109011	0.053126
Lineare Regression	f_regression	6.0	0.666749	0.770271	0.071628	0.043876
	correlation	6.0	0.666749	0.770271	0.071628	0.043876
SVR	correlation	6.0	0.682143	0.759834	0.159869	0.084277
	f_regression	6.0	0.682143	0.759834	0.159869	0.084277
Lineare Regression	mutual_info	6.0	0.687535	0.759937	0.057327	0.041685
K-Nearest Neighbors	mutual_info	6.0	0.762884	0.702398	0.154748	0.079885
SVR	mutual_info	6.0	0.819585	0.656287	0.142198	0.076828

4.2 Fazit

Bei der Anzahl der Features fällt auf, dass bereits eine kleine Auswahl (etwa sechs Features) ausreicht, um eine Vorhersage zu treffen, die sich durch zusätzliche Features nicht weiter verbessern lässt. Dies lässt sich grundsätzlich damit erklären, dass nur ein geringer Teil der Features tatsächlich eine starke Korrelation mit dem Apfelpreis aufweist.

Zwar konnte mit dem Random-Forest-Ansatz ein besseres Ergebnis als mit der linearen Regression erzielt werden, insgesamt unterscheidet sich die Performance der Modelle jedoch nicht signifikant.

Generell zeigt sich beim Random-Forest-Ansatz ($n = 6$) eine höhere Standardabweichung der Metriken über die einzelnen Folds hinweg im Vergleich zur linearen Regression. Letztere weist in Bezug auf den RMSE-Score tendenziell eine geringere Verzerrung auf, ist jedoch insgesamt ungenauer. Der R^2 -Wert zeigt hingegen ein umgekehrtes Verhalten.

Insgesamt lässt sich anhand dieses Beispiels kein klarer Trend in Bezug auf das Bias-Variance-Trade-off zwischen den Verfahren erkennen (auch nicht in Bezug auf die

anderen Verfahren).

5. Unüberwachtes Lernen - Clustern

Auch beim unüberwachten Lernen, also dem Clustern der Städte, wurden mehrere Verfahren ausprobiert. Bei nichtdeterministischen Verfahren wie KMeans, bei denen einzelne Teile des Verfahrens auf Zufall basieren, wurde die Berechnung mit zufälligen Random Seeds insgesamt zehnmal durchgeführt und ein Durchschnittswert für die Scores berechnet. Die Verfahren wurden für verschiedene Clustergrößen zwischen 2 und 40 in Zwischenschritten angewendet.

```
In [58]: from sklearn.cluster import KMeans, SpectralClustering, AgglomerativeClustering
from sklearn.metrics import silhouette_score, calinski_harabasz_score, davies_bouldin_score
from sklearn.preprocessing import StandardScaler

# Selektiere numerische Spalten
numeric_columns = df.select_dtypes(np.number)

# Skalieren der Daten
scaler = StandardScaler()
scaled_numeric = pd.DataFrame(scaler.fit_transform(numeric_columns))

# Entferne Zeilen mit fehlenden Werten
not_na = ~numeric_columns.isna().any(axis=1)
data_to_cluster = scaled_numeric.dropna()

# Initialisiere Speicher für Ergebnisse
results = []

# Definiere Clustering-Methoden
clustering_methods = {
    "KMeans": lambda n_clusters: KMeans(n_clusters=n_clusters, random_state=42),
    "Agglomerative": lambda n_clusters: AgglomerativeClustering(n_clusters=n_clusters),
    "SpectralClustering": lambda n_clusters: SpectralClustering(n_clusters=n_clusters)
}

# Clustergrößen, die evaluiert werden sollen
cluster_sizes = list(range(2, 40, 2))

cluster_dict = {}

# Führe Clustering für jede Methode und Clustergröße durch
for method_name, method in clustering_methods.items():
    for n_clusters in cluster_sizes:
        # Iterationen für Methoden, die Zufälligkeit enthalten
        iterations = 1
        if method_name in ["KMeans", "BisectingKMeans"]:
            iterations = 10
        for i in range(iterations):
```



```

clusters = method(n_clusters).fit_predict(data_to_cluster)

cluster_dict[(method_name, n_clusters)] = clusters

# Berechne Metriken für die Cluster
silhouette_avg = silhouette_score(data_to_cluster, clusters)
calinski_harabasz = calinski_harabasz_score(data_to_cluster, clusters)
davies_bouldin = davies_bouldin_score(data_to_cluster, clusters)
results.append({
    "Method": method_name,
    "Cluster": n_clusters,
    "Silhouette Score": silhouette_avg,
    "Calinski-Harabasz Index": calinski_harabasz,
    "Davies-Bouldin Index": davies_bouldin
})

# Erstelle ein DataFrame mit einem MultiIndex
metrics_df = pd.DataFrame(results)
metrics_df = metrics_df.groupby(["Method", "Cluster"]).mean().reset_index()

metrics_df.set_index(["Method", "Cluster"], inplace=True)
metrics_df.sort_index(inplace=True)

```

5.1 Einfluss der Clustergröße

Das nachfolgende Diagramm zeigt den Verlauf der Metriken für verschiedene Verfahren in Abhängigkeit von der Clustergröße.

```

In [59]: # Erstelle eine Liste der Metriken
metrics = ["Silhouette Score", "Calinski-Harabasz Index", "Davies-Bouldin Index"]

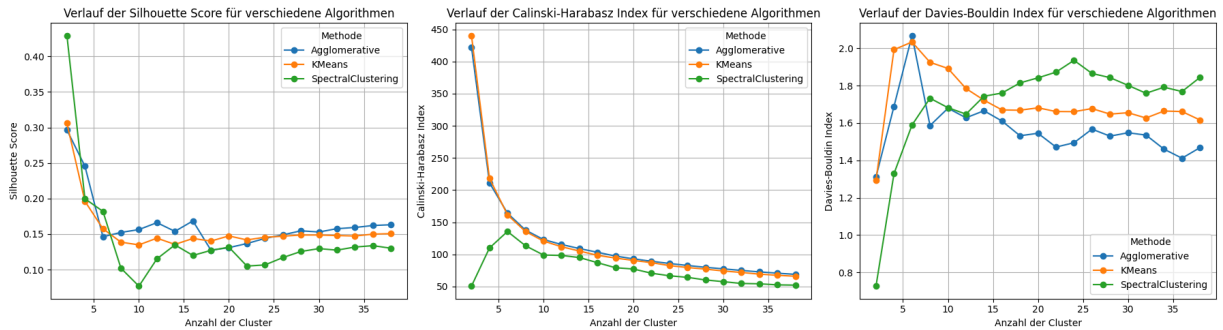
# Erstelle eine Figure mit Subplots
fig, axes = plt.subplots(1, len(metrics), figsize=(18, 5), sharex=True)

# Iteriere über die Metriken und erstelle für jede einen separaten Plot
for i, metric in enumerate(metrics):
    ax = axes[i]
    for method in metrics_df.index.get_level_values("Method").unique():
        subset = metrics_df.loc[method]
        ax.plot(
            subset.index,
            subset[metric],
            marker="o",
            label=method
        )
    ax.set_title(f"Verlauf der {metric} für verschiedene Algorithmen")
    ax.set_xlabel("Anzahl der Cluster")
    ax.set_ylabel(metric)
    ax.legend(title="Methode")
    ax.grid(True)

# Passe das Layout an

```

```
plt.tight_layout()
plt.show()
```



Der Silhouette Score (hoch = gut getrennt & kompakt) fällt bei allen Verfahren nach wenigen Clustern stark ab. Das selbe gilt für den Calinski-Harabasz Index (hoch = gutes Verhältnis zwischen interner Streuung & Clusterabstand). Der Davies-Bouldin Index (niedrig = besser getrennte Cluster) ist bei Agglomerative über weite Bereiche am niedrigsten, während BisectingKMeans konstant schlechtere Werte liefert.

5.2 Clustervisualisierung

Die folgende Grafik zeigt die geografische Verteilung der Cluster auf einer Weltkarte – jeweils für verschiedene Clusterzahlen und Lernverfahren.

```
In [81]: from plotly.subplots import make_subplots
import kaleido
import plotly.express as px

# Cluster-Konfigurationen
cluster_configs = [
    ("KMeans", 2), ("KMeans", 4), ("KMeans", 10),
    ("Agglomerative", 2), ("Agglomerative", 4), ("Agglomerative", 10)
]

# Subplots erstellen
fig = make_subplots(
    rows=3, cols=2,
    subplot_titles=[f"{method} ({n_clusters} Cluster)" for method, n_clusters in cluster_configs],
    specs=[[{"type": "scattergeo"}] * 2] * 3,
    vertical_spacing=0.05, # Abstand zwischen den Zeilen
    horizontal_spacing=0.01 # Abstand zwischen den Spalten
)

# Daten für jede Cluster-Konfiguration hinzufügen
for i, (method, n_clusters) in enumerate(cluster_configs):
    row, col = divmod(i, 2)
    display_data = df[not_na].copy()
    display_data["cluster"] = cluster_dict[(method, n_clusters)].astype("object")

    scatter_geo = px.scatter_geo(
        display_data, lat="latitude", lon="longitude", color="cluster",
        hover_name="city", hover_data=["country", "elevation"],
```

```

        projection="natural earth",
    )
    for trace in scatter_geo.data:
        fig.add_trace(trace, row=row + 1, col=col + 1)

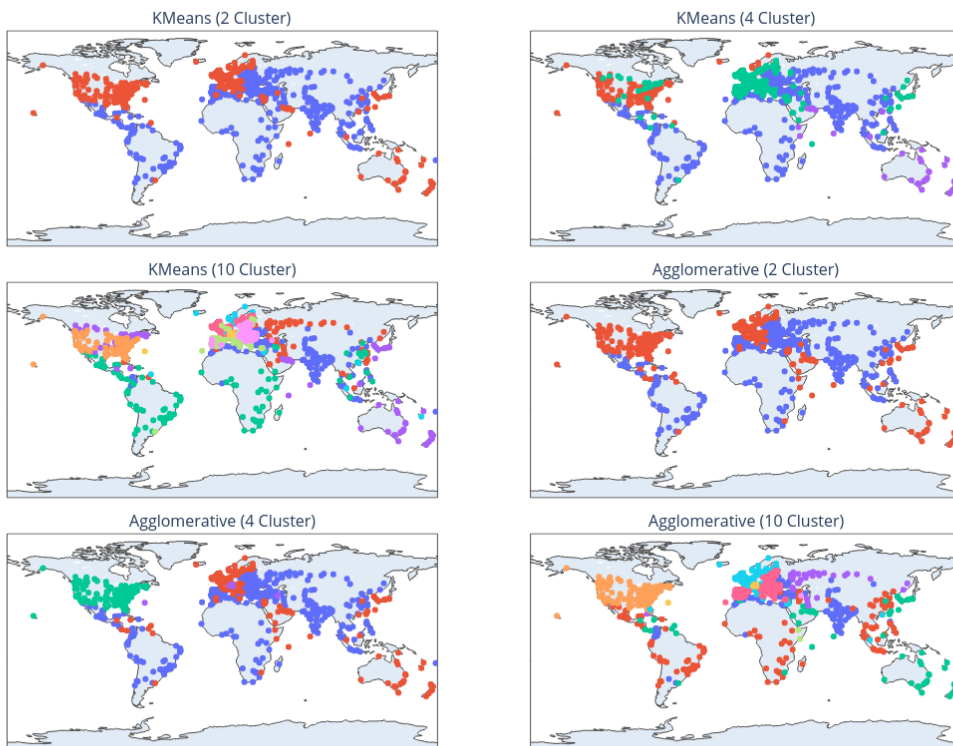
# Layout aktualisieren
fig.update_layout(
    title_text="Clustering der Städte",
    showlegend=False,
)

Image(fig.to_image(format="png", width=1200, height=900))

```

Out[81]:

Clustering der Städte



Generell fällt auf, dass die Cluster wohlhabendere Länder bzw. solche mit unterschiedlichen Lebenshaltungskosten sinnvoll klassifizieren – und das über alle Clustergrößen hinweg. Auch wenn die mathematischen Metriken bei größeren Clusterzahlen tendenziell abnehmen, erscheint die Klassifikation für z.B. $n=10$ dennoch plausibel. Die Verfahren scheinen anhand der Preisstruktur verschiedener Produkte sogar einzelne Länder wie etwa England oder die Schweiz gezielt zu identifizieren (bei $n=10$).

In Bezug auf Aufgabe 1 könnten die Cluster zwar einen Anhaltspunkt für Preisniveaus einzelner Länder bieten, jedoch ist zu beachten, dass der speziell definierte Warenkorb keine maßgeschneiderte Metrik erlaubt. Ein solcher ist vor allem besser interpretierbar, da er nicht unter durch das Lernverfahren bedingten Unregelmäßigkeiten leidet. Aus

diesem Grund wurde entschieden, die Cluster nicht als Grundlage für die Städteauswahl heranzuziehen.

6. Deployment

Es wurde eine umfassende Analyse der Eignung von insgesamt 910 Städten weltweit durchgeführt. Dazu wurde ein individuell auf die Person zugeschnittener Warenkorb definiert, der das Konsumverhalten der Person abbilden soll. Außerdem wurden die Städte anhand ihrer Höhenlage untersucht.

Der Anteil verschiedener Ausgabenkategorien auf verschiedenen Kontinenten ähnlich ist, unterscheidet sich der Gesamtbetrag signifikant.

In Europa liegt nur eine der untersuchten Städte (Insbruk) bei einbeziehung des umlands auf durchschnittlich 1000m höhe, während andere Kontinente deutlich mehr städte in höhenlage aufweisen.

Abschließend wurden die Städte noch nach dem nach den Daten und dem Warenkorb erstellten Kostenindex berechnet